

Codebook

Contents

1	basic	2
1.1	build	2
1.2	default	2
1.3	div	2
1.4	fast	2
1.5	random	2
2	data structure	2
2.1	2d seg	2
2.2	2d seg ptr	2
2.3	bit	3
2.4	bitset	3
2.5	lichao	3
2.6	pbds	4
2.7	range set	4
2.8	sparse table	4
2.9	stl sort	4
2.10	treap	4
2.11	undo dsu	5
2.12	unordered map	5
3	graph	5
3.1	2SAT	5
3.2	bridge	6
3.3	bridge bcc	6
3.4	cut	6
3.5	cut bcc	6
3.6	kosaraju	6
3.7	matching dfs	6
3.8	matching general	7
3.9	matching general weight	7
3.10	prim	8
3.11	spfa	8
4	math	8
4.1	big number	8
4.2	burnside lemma	9
4.3	catalan	9
4.4	count prime	9
4.5	CRT coprime	9
4.6	CRT coprime	10
4.7	CRT not coprime	10
4.8	det	10
4.9	extgcd	10
4.10	factorize	10
4.11	fraction	10
4.12	gaussian elimination	11
4.13	josephus	11
4.14	lucas	11
4.15	math block	11
4.16	milller rabin	11
4.17	phi	12
4.18	phi lemma	12
4.19	primes er	12
4.20	primes linear	12
4.21	two equations	12
5	other	12
5.1	aliens dp	12
5.2	dc dp	12
5.3	digit dp	12
5.4	dinic	13
5.5	geometry	13
5.6	line container	13
5.7	mcmf	13
5.8	mo	14
5.9	number split	14
5.10	sos dp	14
5.11	subset	14
5.12	xor basis	14
6	string	14
6.1	hash	14
6.2	kmp	15
6.3	manacher	15
6.4	string trie	15
6.5	xor trie	15
6.6	z	15
7	tree	16
7.1	hld	16

1 basic

1.1 build

```

1 code tem.cpp
2 ...
3 for i in {A..M}; do cp tem.cpp $i.cpp; done;
4 code *.cpp
5 code run
6
7 寫入以下編譯與執行指令至 run 檔案中：
8 g++ -std=c++20 -DABS -Wall -Wextra -Wshadow $1.cpp -o $1 && ./
9 $1
10 chmod +x run

```

1.2 default

```

1 #include <bits/stdc++.h>
2 #define int long long
3 #define pii pair<int, int>
4 #define F first
5 #define S second
6 #define pb push_back
7 #define all(x) x.begin(), x.end()
8 #define sz(x) (int)x.size()
9 #define FOR(i, a, b) for(int i = a; i <= b; ++i)
10 #define IO ios::sync_with_stdio(0), cin.tie(0)
11 using namespace std;
12 void debug() { cerr << endl; }
13 template <typename T, typename ...U>
14 void debug(T i, U ...j) { cerr << i << ' ', debug(j...); }
15 #define test(...) debug("[ " #__VA_ARGS__ " ]:", __VA_ARGS__)
16
17 signed main() {
18     IO;
19 }
20

```

1.3 div

```

1 ll floor(ll a, ll b) {
2     return a / b - (a < 0 && a % b);
3 }
4 ll ceil(ll a, ll b) {
5     return a / b + (a > 0 && a % b);
6 }
7 a / b < x -> floor(a, b) + 1 <= x
8 a / b <= x -> ceil(a, b) <= x
9 x < a / b -> x <= ceil(a, b) - 1
10 x <= a / b -> x <= floor(a, b)

```

1.4 fast

```

1 #pragma GCC optimize("Ofast,inline,unroll-loops")
2 #pragma GCC target("bmi,bmi2,lzcnt,popcnt,avx2")
3 #define windows_system
4 #ifdef windows_system
5     /* For Windows */
6 inline int getchar_unlocked() { return getchar(); }
7 inline void putchar_unlocked(char _c) { putchar(_c); }
8 #endif
9 inline int in() {
10     int re = 0;
11     char c = getchar_unlocked();
12     while (c == ' ' || c == '\n') c = getchar_unlocked();
13     while (c >= '0' && c <= '9') re = (re << 3) + (re << 1) + c - '0', c = getchar_unlocked();
14     return re;
15 }
16 inline void out(int x) {
17     char str[20];
18     int pos = 0;
19     do {
20         str[pos++] = x % 10 + '0';
21         x /= 10;
22     } while (x);
23     for (int i = pos - 1; i >= 0; i--) putchar_unlocked(str[i]);
24     putchar_unlocked('\n');
25 }

```

1.5 random

```

1 mt19937 seed(chrono::steady_clock::now().time_since_epoch().
2     count());
3 inline int rnd(int l, int r) {
4     return uniform_int_distribution<int>(l, r)(seed);
5 }
6 inline double drnd(double l, double r) {
7     return uniform_real_distribution<double>(l, r)(seed);
8 }

```

2 data structure

2.1 2d seg

```

1 struct Node {
2     int x;
3     Node(int _x = 0) : x(_x) {}
4 };
5 Node combine(Node a, Node b) {
6     return Node(max(a.x, b.x));
7 }
8 struct Segment_Tree {
9     struct Segment_Tree2 {
10         vector<Node> seg2;
11         int n;
12         Segment_Tree2(int _n) : n(_n), seg2(4 * _n + 5) {}
13         void update2(int node, int l, int r, int id, int va) {
14             if (l == r) {
15                 seg2[node].x = va;
16                 return;
17             }
18             int mid = (l + r) >> 1;
19             if (id <= mid)
20                 update2((node << 1), l, mid, id, va);
21             else
22                 update2(((node << 1) | 1), mid + 1, r, id, va);
23             seg2[node] = combine(seg2[(node << 1)], seg2[(node << 1) | 1]);
24         }
25         Node query2(int node, int l, int r, int ql, int qr) {
26             if (ql <= l && r <= qr) return seg2[node];
27             int mid = (l + r) >> 1;
28             if (qr <= mid)
29                 return query2((node << 1), l, mid, ql, qr);
30             else if (mid + 1 <= ql)
31                 return query2(((node << 1) | 1), mid + 1, r, ql, qr);
32             else
33                 return combine(query2((node << 1), l, mid, ql, qr),
34                     query2(((node << 1) | 1), mid + 1, r, ql, qr));
35         }
36     };
37     vector<Segment_Tree2> seg;
38     int n;
39     Segment_Tree(int _n) : n(_n), seg(4 * _n + 5, Segment_Tree2(_n)) {}
40     void update(int node, int l, int r, int x, int y, int va) {
41         seg[node].update2(1, 1, n, y, va);
42         if (l == r) return;
43         int mid = (l + r) >> 1;
44         if (x <= mid)
45             update((node << 1), l, mid, x, y, va);
46         else
47             update(((node << 1) | 1), mid + 1, r, x, y, va);
48     }
49     Node query(int node, int l, int r, int xl, int xr, int yl,
50         int yr) {
51         if (xl <= l && r <= xr) return seg[node].query2(1, 1, n, yl, yr);
52         int mid = (l + r) >> 1;
53         if (xr <= mid)
54             return query((node << 1), l, mid, xl, xr, yl, yr);
55         else if (mid + 1 <= xl)
56             return query(((node << 1) | 1), mid + 1, r, xl, xr, yl, yr);
57         else
58             return combine(query((node << 1), l, mid, xl, xr, yl, yr),
59                 query(((node << 1) | 1), mid + 1, r, xl, xr, yl, yr));
60     }
61 }

```

2.2 2d seg ptr

```

1 // CSES Forest Queries II TLE
2 struct Node2 {
3     int val; Node2 *lc, *rc;
4     Node2 () {}

```

```

5     val = 0;
6     lc = rc = nullptr;
7 }
8 };
9 struct Node1 {
10     Node2 *c; Node1 *lc, *rc;
11     Node1 () {
12         c = nullptr;
13         lc = rc = nullptr;
14     }
15 };
16 int N;
17 int Val2(Node2 *n2) { return (n2 ? n2->val : 011); }
18 void pull2(Node2 *n2) { n2->val = Val2(n2->lc) + Val2(n2->rc);
19 }
20 void update2(Node2 *n2, int l, int r, int id, int v) {
21     if (l == r) {
22         n2->val += v; return;
23     }
24     int mid = (l + r) >> 1;
25     if (id <= mid) {
26         if (!n2->lc) n2->lc = new Node2();
27         update2(n2->lc, l, mid, id, v);
28     } else {
29         if (!n2->rc) n2->rc = new Node2();
30         update2(n2->rc, mid + 1, r, id, v);
31     }
32     pull2(n2);
33 }
34 void update1(Node1 *n1, int l, int r, int qx, int qy, int v) {
35     if (!n1->c) n1->c = new Node2();
36     update2(n1->c, l, N, qy, v);
37     if (l == r) return;
38     int mid = (l + r) >> 1;
39     if (qx <= mid) {
40         if (!n1->lc) n1->lc = new Node1();
41         update1(n1->lc, l, mid, qx, qy, v);
42     } else {
43         if (!n1->rc) n1->rc = new Node1();
44         update1(n1->rc, mid + 1, r, qx, qy, v);
45     }
46 }
47 int query2(Node2 *n2, int l, int r, int ql, int qr) {
48     if (!n2) return 0;
49     if (ql <= l && r <= qr) return n2->val;
50     int mid = (l + r) >> 1;
51     if (qr <= mid)
52         return query2(n2->lc, l, mid, ql, qr);
53     else if (mid + 1 <= ql)
54         return query2(n2->rc, mid + 1, r, ql, qr);
55     else
56         return query2(n2->lc, l, mid, ql, qr) + query2(n2->rc, mid
57             + 1, r, ql, qr);
58 }
59 int query1(Node1 *n1, int l, int r, int qx1, int qxr, int qyl,
60     int qyr) {
61     if (!n1) return 0;
62     if (qx1 <= l && r <= qxr) return query2(n1->c, l, N, qyl, qyr
63         );
64     int mid = (l + r) >> 1;
65     if (qxr <= mid)
66         return query1(n1->lc, l, mid, qx1, qxr, qyl, qyr);
67     else if (mid + 1 <= qx1)
68         return query1(n1->rc, mid + 1, r, qx1, qxr, qyl, qyr);
69     else
70         return query1(n1->lc, l, mid, qx1, qxr, qyl, qyr) + query1(
71             n1->rc, mid + 1, r, qx1, qxr, qyl, qyr);
72 }
73 bool b[N + 5][N + 5];
74 signed main() {
75     Node1 *root = new Node1();
76     int n, q;
77     cin >> n >> q;
78     N = n;
79     char ch;
80     for (int i = 1; i <= n; i++) {
81         for (int j = 1; j <= n; j++) {
82             cin >> ch;
83             if (ch == '*') {
84                 b[i][j] = 1;
85                 update1(root, 1, N, i, j, 1);
86             } else
87                 b[i][j] = 0;
88         }
89     }
90     int p;
91     int x, y, xl, xr, yl, yr;
92     while (q--) {
93         cin >> p;
94         if (p == 1) {
95             cin >> x >> y;
96             if (b[x][y]) {
97                 b[x][y] = 0;
98                 update1(root, 1, N, x, y, -1);
99             } else {
100                 b[x][y] = 1;
101                 update1(root, 1, N, x, y, +1);

```

```

97     }
98     } else {
99         cin >> xl >> yl >> xr >> yr;
100         cout << query1(root, 1, N, xl, xr, yl, yr) << "\n";
101     }
102 }
103 return 0;
104 }

```

2.3 bit

```

1 struct BIT {
2     int N;
3     vector<int> info;
4     BIT(int n) : N(n), info(n + 1) {}
5     void update(int idx, int val) {
6         for (int i = idx; i <= N; i += i & -i)
7             info[i] += val;
8     }
9     int query(int idx) {
10         int res = 0;
11         for (int i = idx; i > 0; i -= i & -i)
12             res += info[i];
13         return res;
14     }
15     int kth(int k) { // kth small
16         int res = 0;
17         for (int i = 1 << __lg(N); i > 0; i >= 1) {
18             if (res + i <= N && info[res + i] < k) {
19                 res += i;
20                 k -= info[res];
21             }
22         }
23         return res + 1;
24     }
25 };

```

2.4 bitset

```

1 #pragma GCC target("popcnt")
2 // 15 = 1111
3 bitset<10> bit(15); // 宣告 長度 = 10 的 bitset
4 cout << bit.count() << "\n"; // 回傳 幾個 1
5 cout << bit.none() << "\n"; // 回傳 是否全為 0
6 cout << bit.any() << "\n"; // 回傳 是否有 1
7 cout << bit._Find_first() << "\n"; // 回傳 第一個 bit = 1 的位
8 // 置
9 int x = 2;
10 cout << bit._Find_next(x) << "\n"; // 回傳 x 後 第一個 bit = 1
11 // 的位置

```

2.5 lichao

```

1 // MIN LeeChao Segment Tree
2 struct line {
3     int m, k;
4     line *lc, *rc;
5     line() { m = 0, k = INT_MAX, lc = nullptr, rc = nullptr; }
6     line(int _m, int _k) { m = _m, k = _k, lc = nullptr, rc =
7         nullptr; }
8     int calc(int x) {
9         return m * x + k;
10    }
11    void push(int l, int r) {
12        if (l == r) return;
13        if (!lc) lc = new line();
14        if (!rc) rc = new line();
15    }
16 };
17 struct LeeChao {
18     line* root;
19     LeeChao() : root(new line()) {}
20     void update(line* node, int l, int r, line add) {
21         if (l == r) {
22             if (add.calc(l) < node->calc(l)) {
23                 swap(add.m, node->m);
24                 swap(add.k, node->k);
25             }
26             return;
27         }
28         // 預設 seg[node] 的斜率 < add 的斜率
29         if (node->m > add.m) {
30             swap(add.m, node->m);
31             swap(add.k, node->k);
32         }
33         node->push(l, r);

```

```

33 int mid = (l + r) >> 1;
34 // 將小的 f(x) 留下
35 if (node->calc(mid) > add.calc(mid)) {
36     swap(add.m, node->m);
37     swap(add.k, node->k);
38     update(node->rc, mid + 1, r, add);
39 } else {
40     update(node->lc, l, mid, add);
41 }
42 }
43 int query(line* node, int l, int r, int x) {
44     if (node->k == INT_MAX) return INT_MAX;
45     if (l == r) return node->calc(x);
46     int mid = (l + r) >> 1;
47     if (x <= mid)
48         return min(node->calc(x), query(node->lc, l, mid, x));
49     else
50         return min(node->calc(x), query(node->rc, mid + 1, r, x));
51 }
52 void update(line l) { update(root, 1, N, l); }
53 int query(int x) { return query(root, 1, N, x); }
54 };
55 // Lee-Chao S;

```

2.6 pbds

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 void solve() {
5     tree<int, null_type, greater<int>, rb_tree_tag,
6         tree_order_statistics_node_update> bst;
7     bst.insert(1);
8     int a = *bst.find_by_order(2); // 第 k + 1 大的數
9     int b = bst.order_of_key(2); // 有幾個數 > k
10    // -----
11    tree<int, null_type, less<int>, rb_tree_tag,
12        tree_order_statistics_node_update> bst;
13    bst.insert(1);
14    int a = *bst.find_by_order(2); // 第 k + 1 小的數
15    int b = bst.order_of_key(2); // 有幾個數 < k
16 }

```

2.7 range set

```

1 struct RangeSet { // [L, r)
2     set<pii> st;
3     void cut(int x) {
4         auto it = st.lower_bound({x + 1, -1});
5         if (it == st.begin()) return;
6         auto [l, r] = *prev(it);
7         if (l >= x || x >= r) return;
8         st.erase(prev(it));
9         st.insert({l, x});
10        st.insert({x, r});
11    }
12    vector<pii> split(int l, int r) {
13        // remove and return ranges in [L, r)
14        cut(l), cut(r);
15        vector<pii> res;
16        while (true) {
17            auto it = st.lower_bound({l, -1});
18            if (it == st.end() || r <= it->first) break;
19            res.push_back(*it), st.erase(it);
20        }
21        return res;
22    }
23    void insert(int l, int r) {
24        // add a range [L, r), [L, r) not in st
25        auto it = st.lower_bound({l, r});
26        if (it != st.begin() && prev(it)->second == l)
27            l = prev(it)->first, st.erase(prev(it));
28        if (it != st.end() && r == it->first)
29            r = it->second, st.erase(it);
30        st.insert({l, r});
31    }
32    bool count(int x) {
33        auto it = st.lower_bound({x + 1, -1});
34        return it != st.begin() && prev(it)->first <= x && x < prev
35            (it)->second;
36    }
37 };

```

2.8 sparse table

```

1 int a[N];
2 struct Sparse_table {
3     int n;
4     vector<vector<int>>> st;
5     Sparse_table(int _n) : n(_n), st(_n + 1, vector<int>(__lg(_n)
6         + 1)) {
7         for (int i = 1; i <= n; i++) st[i][0] = a[i];
8         for (int lg = 1; lg <= __lg(n); lg++) {
9             int len = 1 << (lg - 1);
10            for (int i = 1; i + len <= n; i++)
11                st[i][lg] = max(st[i][lg - 1], st[i + len][lg - 1]);
12        }
13    }
14    int query(int l, int r) {
15        int lg = __lg(r - l + 1);
16        int len = 1 << lg;
17        return max(st[l][lg], st[r - len + 1][lg]);
18    }
19 };

```

2.9 stl sort

```

1 struct Data {
2     int x, y;
3 };
4 // priority_queue
5 struct cmp {
6     bool operator()(Data a, Data b) {
7         return a.x < b.x;
8     }
9 };
10 priority_queue<Data, vector<Data>, cmp> pq;
11 // set
12 struct cmp {
13     bool operator()(Data p, Data q) {
14         return p.x < q.x;
15     }
16 };
17 set<Data, cmp> st;

```

2.10 treap

```

1 struct node {
2     int data, sz;
3     node *l, *r;
4     node(int k) : data(k), sz(1), l(0), r(0) {}
5     void up() {
6         sz = 1;
7         if (l) sz += l->sz;
8         if (r) sz += r->sz;
9     }
10    void down() {}
11 };
12 int sz(node *a) { return a ? a->sz : 0; }
13 node *merge(node *a, node *b) {
14     if (!a || !b) return a ? a : b;
15     if (rand() % (sz(a) + sz(b)) < sz(a))
16         return a->down(), a->r = merge(a->r, b), a->up(), a;
17     return b->down(), b->l = merge(a, b->l), b->up(), b;
18 }
19 void split1(node *o, node *&a, node *&b, int k) {
20     if (!o) return a = b = 0, void();
21     o->down();
22     if (o->data <= k)
23         a = o, split1(o->r, a->r, b, k), a->up();
24     else
25         b = o, split1(o->l, a, b->l, k), b->up();
26 }
27 void split2(node *o, node *&a, node *&b, int k) {
28     if (sz(o) <= k) return a = o, b = 0, void();
29     o->down();
30     if (sz(o->l) + 1 <= k)
31         a = o, split2(o->r, a->r, b, k - sz(o->l) - 1);
32     else
33         b = o, split2(o->l, a, b->l, k);
34     o->up();
35 }
36 node *kth(node *o, int k) {
37     if (k <= sz(o->l)) return kth(o->l, k);
38     if (k == sz(o->l) + 1) return o;
39     return kth(o->r, k - sz(o->l) - 1);
40 }
41 int Rank(node *o, int key) {
42     if (!o) return 0;
43     if (o->data < key)
44         return sz(o->l) + 1 + Rank(o->r, key);
45     else
46         return Rank(o->l, key);
47 }
48 bool erase(node *o, int k) {
49     if (!o) return 0;

```

```

50 if (o->data == k) {
51     node *t = o;
52     o->down(), o = merge(o->l, o->r);
53     delete t;
54     return 1;
55 }
56 node *&t = k < o->data ? o->l : o->r;
57 return erase(t, k) ? o->up(), 1 : 0;
58 }
59 void insert(node *&o, int k) {
60     node *a, *b;
61     split(o, a, b, k);
62     o = merge(a, merge(new node(k), b));
63 }
64 void interval(node *&o, int l, int r) {
65     node *a, *b, *c;
66     split2(o, a, b, l - 1), split2(b, b, c, r);
67     // operate
68     o = merge(a, merge(b, c));
69 }

```

2.11 undo dsu

```

1 struct DSU {
2     vector<int> parent, size, tag;
3     vector<tuple<int, int, int, int, int>> history;
4     DSU(int n) {
5         parent.resize(n + 1);
6         size.resize(n + 1, 1);
7         tag.resize(n + 1, 0);
8         for (int i = 1; i <= n; i++) parent[i] = i;
9     }
10    int find(int x) {
11        if (parent[x] == x) return x;
12        return find(parent[x]);
13    }
14    bool merge(int x, int y) {
15        x = find(x), y = find(y);
16        if (x == y) return false;
17        if (size[x] < size[y]) swap(x, y);
18        history.pb({x, y, parent[y], tag[x], size[x]});
19        parent[y] = x;
20        size[x] += size[y];
21        return true;
22    }
23    void add_tag(int head, int val) {
24        tag[head] += val;
25    }
26    void undo() {
27        auto [px, py, parent_py, base, sz_x] = history.back();
28        parent[py] = parent_py;
29        size[px] = sz_x;
30        int delta = tag[px] - base;
31        tag[py] += delta;
32        history.pop_back();
33    }
34 };

```

2.12 unordered map

```

1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t a) const {
9         static const uint64_t FIXED_RANDOM = chrono::steady_clock::
10            now().time_since_epoch().count();
11         return splitmix64(i + FIXED_RANDOM);
12     }
13 };
14 unordered_map <int, int, custom_hash> m1;

```

3 graph

3.1 2SAT

```

1 struct TWO_SAT {
2     int n, N;
3     vector<vector<int>> G, rev_G;
4     deque<bool> used;
5     vector<int> order, comp;
6     deque<bool> assignment;

```

```

7     void init(int _n) {
8         n = _n;
9         N = _n * 2;
10        G.resize(N + 5);
11        rev_G.resize(N + 5);
12    }
13    void dfs1(int v) {
14        used[v] = true;
15        for (int u : G[v]) {
16            if (!used[u])
17                dfs1(u);
18        }
19        order.push_back(v);
20    }
21    void dfs2(int v, int c1) {
22        comp[v] = c1;
23        for (int u : rev_G[v]) {
24            if (comp[u] == -1)
25                dfs2(u, c1);
26        }
27    }
28    bool solve() {
29        order.clear();
30        used.assign(N, false);
31        for (int i = 0; i < N; ++i) {
32            if (!used[i])
33                dfs1(i);
34        }
35        comp.assign(N, -1);
36        for (int i = 0, j = 0; i < N; ++i) {
37            int v = order[N - i - 1];
38            if (comp[v] == -1)
39                dfs2(v, j++);
40        }
41        assignment.assign(n, false);
42        for (int i = 0; i < N; i += 2) {
43            if (comp[i] == comp[i + 1])
44                return false;
45            assignment[i / 2] = (comp[i] > comp[i + 1]);
46        }
47        return true;
48    }
49    void add_disjunction(int a, bool na, int b, bool nb) { // A
50        // or B
51        // na means whether a is negative or not
52        // nb means whether b is negative or not
53        a = 2 * a ^ na;
54        b = 2 * b ^ nb;
55        int neg_a = a ^ 1;
56        int neg_b = b ^ 1;
57        G[neg_a].push_back(b);
58        G[neg_b].push_back(a);
59        rev_G[b].push_back(neg_a);
60        rev_G[a].push_back(neg_b);
61        return;
62    }
63    void get_result(vector<int>& res) {
64        res.clear();
65        for (int i = 0; i < n; i++)
66            res.push_back(assignment[i]);
67    }
68    /* CSES Giant Pizza
69    3 5
70    + 1 + 2
71    - 1 + 3
72    + 4 - 2
73    - + + + -
74    */
75    int main() {
76        int n, m;
77        cin >> n >> m;
78        TWO_SAT E;
79        E.init(m);
80        char c1, c2;
81        int in1, in2;
82        for (int i = 0; i < n; i++) {
83            cin >> c1 >> in1;
84            cin >> c2 >> in2;
85            E.add_disjunction(in1 - 1, c1 == '-', in2 - 1, c2 == '-')
86                ;
87        }
88        bool able = E.solve();
89        if (able) {
90            vector<int> ans;
91            E.get_result(ans);
92            for (int i : ans)
93                cout << (i == true ? '+' : '-') << ' ';
94            cout << '\n';
95        } else {
96            cout << "IMPOSSIBLE\n";
97        }
98        return 0;
99    }

```

3.2 bridge

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<pair<int, int>> bridge;
6 void dfs(int v, int p) {
7     vis[v] = true;
8     low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
9     for (int u : G[v]) {
10         if (u == p) continue;
11         if (!vis[u]) {
12             // (v, u) 是樹邊
13             dfs(u, v);
14             low[v] = min(low[v], low[u]);
15         } else {
16             // (v, u) 是回邊
17             low[v] = min(low[v], depth[u]);
18         }
19     }
20     // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
21     // (root, -1) 不算
22     if (low[v] == depth[v] && p != -1)
23         bridge.push_back({v, p});
24 }

```

3.3 bridge bcc

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<vector<int>> bcc;
6 stack<int> stk;
7 void dfs(int v, int p) {
8     stk.push(v);
9     vis[v] = true;
10    low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
11    for (int u : G[v]) {
12        if (u == p) continue;
13        if (!vis[u]) {
14            // (v, u) 是樹邊
15            dfs(u, v);
16            low[v] = min(low[v], low[u]);
17        } else {
18            // (v, u) 是回邊
19            low[v] = min(low[v], depth[u]);
20        }
21    }
22    // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
23    if (low[v] == depth[v]) {
24        bcc.emplace_back();
25        while (stk.top() != v) {
26            bcc.back().push_back(stk.top());
27            stk.pop();
28        }
29        bcc.back().push_back(stk.top());
30        stk.pop();
31    }
32 }

```

3.4 cut

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N], cut[N];
5 void dfs(int v, int p) {
6     vis[v] = true;
7     int child = 0;
8     low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
9     for (int u : G[v]) {
10         if (u == p) continue;
11         if (!vis[u]) {
12             // (v, u) 是樹邊
13             dfs(u, v);
14             child++;
15             low[v] = min(low[v], low[u]);
16             // u 無法在不經過父邊的情況走到 v 的祖先
17             if (low[u] >= depth[v] && p != -1)
18                 cut[v] = true;
19         } else {
20             // (v, u) 是回邊
21             low[v] = min(low[v], depth[u]);
22         }
23     }
24     // 根節點有超過 2 個子節點

```

```

25     if (p == -1 && child >= 2)
26         cut[v] = true;
27 }

```

3.5 cut bcc

```

1 const int N = 200005;
2 vector<int> G[N];
3 int low[N], depth[N];
4 bool vis[N];
5 vector<vector<int>> bcc;
6 stack<int> stk;
7 void dfs(int v, int p) {
8     stk.push(v);
9     vis[v] = true;
10    low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
11    for (int u : G[v]) {
12        if (u == p) continue;
13        if (!vis[u]) {
14            // (v, u) 是樹邊
15            dfs(u, v);
16            low[v] = min(low[v], low[u]);
17            // u 無法在不經過父邊的情況走到 v 的祖先
18            if (low[u] >= depth[v]) {
19                bcc.emplace_back();
20                while (stk.top() != u) {
21                    bcc.back().push_back(stk.top());
22                    stk.pop();
23                }
24                bcc.back().push_back(stk.top());
25                stk.pop();
26                bcc.back().push_back(v);
27            }
28        } else {
29            // (v, u) 是回邊
30            low[v] = min(low[v], depth[u]);
31        }
32    }
33 }

```

3.6 kosaraju

```

1 const int N = 200005;
2 vector<int> G[N], rev_G[N], order;
3 bool vis[N]; int scc[N];
4 void rev_dfs(int v) {
5     vis[v] = true;
6     for (int u : rev_G[v])
7         if (!vis[u])
8             rev_dfs(u);
9     order.push_back(v);
10 }
11 void dfs(int v, int block) {
12     scc[v] = block;
13     for (int u : G[v])
14         if (scc[u] == -1)
15             dfs(u, block);
16 }
17 void kosaraju(int n) {
18     for (int i = 1; i <= n; i++)
19         if (!vis[i])
20             rev_dfs(i);
21     int scc_cnt = 0;
22     fill(scc, scc + N, -1);
23     for (int i = n - 1; i >= 0; i--)
24         if (scc[order[i]] == -1)
25             dfs(order[i], ++scc_cnt);
26 }

```

3.7 matching dfs

```

1 const int N = 2005;
2 vector<int> G[N];
3 struct Matching {
4     int n;
5     vector<bool> vis; vector<int> match;
6     Matching(int _n) : n(_n), vis(_n + 1), match(_n + 1, -1) {}
7     bool dfs(int v) {
8         vis[v] = true;
9         for (int u : G[v])
10             if (match[u] == -1 || (!vis[match[u]] && dfs(match[u])))
11                 {
12                     match[u] = v;
13                     return true;
14                 }
15         return false;
16     }
17 }

```

```

16 int solve() {
17     int cnt = 0;
18     for (int i = 1; i <= n; i++) {
19         vis.assign(n + 1, 0);
20         if (dfs(i)) cnt++;
21     }
22     return cnt;
23 }
24 };

```

3.8 matching general

```

1 // O(V^3)
2 // if graph.match[i] == n, node i is not matched
3 // else, i is matched with graph.match[i]
4 struct Matching { // 0-based
5     int fa[N], pre[N], match[N], s[N], v[N], n, tk;
6     vector<int> g[N];
7     queue<int> q;
8     int Find(int u) {
9         return u == fa[u] ? u : fa[u] = Find(fa[u]);
10    }
11    int lca(int x, int y) {
12        tk++;
13        x = Find(x), y = Find(y);
14        for (; ; swap(x, y)) {
15            if (x != n) {
16                if (v[x] == tk) return x;
17                v[x] = tk;
18                x = Find(pre[match[x]]);
19            }
20        }
21    }
22    void blossom(int x, int y, int l) {
23        while (Find(x) != l) {
24            pre[x] = y, y = match[x];
25            if (s[y] == 1) q.push(y), s[y] = 0;
26            if (fa[x] == x) fa[x] = l;
27            if (fa[y] == y) fa[y] = l;
28            x = pre[y];
29        }
30    }
31    bool bfs(int r) {
32        for (int i = 0; i <= n; ++i) fa[i] = i, s[i] = -1;
33        while (!q.empty()) q.pop();
34        q.push(r);
35        s[r] = 0;
36        while (!q.empty()) {
37            int x = q.front(); q.pop();
38            for (int u : g[x]) {
39                if (s[u] == -1) {
40                    pre[u] = x, s[u] = 1;
41                    if (match[u] == n) {
42                        for (int a = u, b = x, last; b != n; a = last, b = pre[a])
43                            last = match[b], match[b] = a, match[a] = b;
44                        return true;
45                    }
46                    q.push(match[u]);
47                    s[match[u]] = 0;
48                } else if (!s[u] && Find(u) != Find(x)) {
49                    int l = lca(u, x);
50                    blossom(x, u, l);
51                    blossom(u, x, l);
52                }
53            }
54        }
55        return false;
56    }
57    int solve() {
58        int res = 0;
59        for (int x = 0; x < n; ++x) {
60            if (match[x] == n) res += bfs(x);
61        }
62        return res;
63    }
64    void init(int _n) {
65        n = _n, tk = 0;
66        for (int i = 0; i <= n; ++i) match[i] = pre[i] = n;
67        for (int i = 0; i < n; ++i) g[i].clear(), v[i] = 0;
68    }
69    void add_edge(int u, int v) {
70        g[u].push_back(v), g[v].push_back(u);
71    }
72 };

```

3.9 matching general weight

```

1 // O(V^3)
2 // if graph.match[i] == 0, node i is not matched
3 // else, i is matched with graph.match[i]

```

```

4 struct WeightGraph { // 1-based
5     static const int inf = INT_MAX;
6     static const int maxn = 514;
7     struct edge {
8         int u, v, w;
9         edge(){}
10        edge(int u, int v, int w): u(u), v(v), w(w) {}
11    };
12    int n, n_x;
13    edge g[maxn * 2][maxn * 2];
14    int lab[maxn * 2];
15    int match[maxn * 2], slack[maxn * 2], st[maxn * 2], pa[maxn * 2];
16    int flo_from[maxn * 2][maxn + 1], S[maxn * 2], vis[maxn * 2];
17    vector<int> flo[maxn * 2];
18    queue<int> q;
19    int e_delta(const edge &e) { return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
20    void update_slack(int u, int x) { if (!slack[x] || e_delta(g[u][x]) < e_delta(g[slack[x]][x])) slack[x] = u; }
21    void set_slack(int x) {
22        slack[x] = 0;
23        for (int u = 1; u <= n; ++u)
24            if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
25                update_slack(u, x);
26    }
27    void q_push(int x) {
28        if (x <= n) q.push(x);
29        else for (size_t i = 0; i < flo[x].size(); i++) q_push(flo[x][i]);
30    }
31    void set_st(int x, int b) {
32        st[x] = b;
33        if (x > n) for (size_t i = 0; i < flo[x].size(); ++i)
34            set_st(flo[x][i], b);
35    }
36    int get_pr(int b, int xr) {
37        int pr = find(flo[b].begin(), flo[b].end(), xr) - flo[b].begin();
38        if (pr % 2 == 1) {
39            reverse(flo[b].begin() + 1, flo[b].end());
40            return (int)flo[b].size() - pr;
41        }
42        return pr;
43    }
44    void set_match(int u, int v) {
45        match[u] = g[u][v].v;
46        if (u <= n) return;
47        edge e = g[u][v];
48        int xr = flo_from[u][e.u], pr = get_pr(u, xr);
49        for (int i = 0; i < pr; ++i) set_match(flo[u][i], flo[u][i ^ 1]);
50        set_match(xr, v);
51        rotate(flo[u].begin(), flo[u].begin() + pr, flo[u].end());
52    }
53    void augment(int u, int v) {
54        for (; ) {
55            int xnv = st[match[u]];
56            set_match(u, v);
57            if (!xnv) return;
58            set_match(xnv, st[pa[xnv]]);
59            u = st[pa[xnv]], v = xnv;
60        }
61    }
62    int get_lca(int u, int v) {
63        static int t = 0;
64        for (++t; u || v; swap(u, v)) {
65            if (u == 0) continue;
66            if (vis[u] == t) return u;
67            vis[u] = t;
68            u = st[match[u]];
69            if (u) u = st[pa[u]];
70        }
71        return 0;
72    }
73    void add_blossom(int u, int lca, int v) {
74        int b = n + 1;
75        while (b <= n_x && st[b]) ++b;
76        if (b > n_x) ++n_x;
77        lab[b] = 0, S[b] = 0;
78        match[b] = match[lca];
79        flo[b].clear();
80        flo[b].push_back(lca);
81        for (int x = u, y, x != lca; x = st[pa[y]])
82            flo[b].push_back(x), flo[b].push_back(y = st[match[x]]), q_push(y);
83        reverse(flo[b].begin() + 1, flo[b].end());
84        for (int x = v, y, x != lca; x = st[pa[y]])
85            flo[b].push_back(x), flo[b].push_back(y = st[match[x]]), q_push(y);
86        set_st(b, b);
87        for (int x = 1; x <= n_x; ++x) g[b][x].w = g[x][b].w = 0;
88        for (int x = 1; x <= n; ++x) flo_from[b][x] = 0;
89        for (size_t i = 0; i < flo[b].size(); ++i) {
90            int xs = flo[b][i];
91            for (int x = 1; x <= n_x; ++x)

```



```

91     if (g[b][x].w == 0 || e_delta(g[xs][x]) < e_delta(g[b][
92         x]))
93         g[b][x] = g[xs][x], g[x][b] = g[x][xs];
94     for (int x = 1; x <= n; ++x)
95         if (flo_from[xs][x]) flo_from[b][x] = xs;
96     }
97     set_slack(b);
98 }
99 void expand_blossom(int b) {
100     for (size_t i = 0; i < flo[b].size(); ++i)
101         set_st(flo[b][i], flo[b][i]);
102     int xr = flo_from[b][g[b][pa[b]].u], pr = get_pr(b, xr);
103     for (int i = 0; i < pr; i += 2) {
104         int xs = flo[b][i], xns = flo[b][i + 1];
105         pa[xs] = g[xns][xs].u;
106         S[xs] = 1, S[xns] = 0;
107         slack[xs] = 0, set_slack(xns);
108         q_push(xns);
109     }
110     S[xr] = 1, pa[xr] = pa[b];
111     for (size_t i = pr + 1; i < flo[b].size(); ++i) {
112         int xs = flo[b][i];
113         S[xs] = -1, set_slack(xs);
114     }
115     st[b] = 0;
116 }
117 bool on_found_edge(const edge &e) {
118     int u = st[e.u], v = st[e.v];
119     if (S[v] == -1) {
120         pa[v] = e.u, S[v] = 1;
121         int nu = st[match[v]];
122         slack[v] = slack[nu] = 0;
123         S[nu] = 0, q_push(nu);
124     } else if (S[v] == 0) {
125         int lca = get_lca(u, v);
126         if (!lca)
127             return augment(u, v), augment(v, u), true;
128         else
129             add_blossom(u, lca, v);
130     }
131     return false;
132 }
133 bool matching() {
134     memset(S + 1, -1, sizeof(int) * n_x);
135     memset(slack + 1, 0, sizeof(int) * n_x);
136     q = queue<int>();
137     for (int x = 1; x <= n_x; ++x)
138         if (st[x] == x && !match[x])
139             pa[x] = 0, S[x] = 0, q_push(x);
140     if (q.empty()) return false;
141     for (; ) {
142         while (q.size()) {
143             int u = q.front(); q.pop();
144             if (S[st[u]] == 1) continue;
145             for (int v = 1; v <= n; ++v)
146                 if (g[u][v].w > 0 && st[u] != st[v]) {
147                     if (e_delta(g[u][v]) == 0) {
148                         if (on_found_edge(g[u][v]))
149                             return true;
150                     } else
151                         update_slack(u, st[v]);
152                 }
153         }
154         int d = inf;
155         for (int b = n + 1; b <= n_x; ++b)
156             if (st[b] == b && S[b] == 1) d = min(d, lab[b] / 2);
157         for (int x = 1; x <= n_x; ++x)
158             if (st[x] == x && slack[x]) {
159                 if (S[x] == -1) d = min(d, e_delta(g[slack[x]][x]));
160                 else if (S[x] == 0) d = min(d, e_delta(g[slack[x]][x]) / 2);
161             }
162         for (int u = 1; u <= n; ++u) {
163             if (S[st[u]] == 0) {
164                 if (lab[u] <= d) return 0;
165                 lab[u] -= d;
166             } else if (S[st[u]] == 1) lab[u] += d;
167         }
168         for (int b = n + 1; b <= n_x; ++b)
169             if (st[b] == b) {
170                 if (S[st[b]] == 0) lab[b] += d * 2;
171                 else if (S[st[b]] == 1) lab[b] -= d * 2;
172             }
173         q = queue<int>();
174         for (int x = 1; x <= n_x; ++x)
175             if (st[x] == x && slack[x] && st[slack[x]] != x &&
176                 e_delta(g[slack[x]][x]) == 0)
177                 if (on_found_edge(g[slack[x]][x]))
178                     return true;
179         for (int b = n + 1; b <= n_x; ++b)
180             if (st[b] == b && S[b] == 1 && lab[b] == 0)
181                 expand_blossom(b);
182     }
183     return false;
184 }
185 pair<long long, int> solve() {
186     memset(match + 1, 0, sizeof(int) * n);

```

```

185     n_x = n;
186     int n_matches = 0;
187     long long tot_weight = 0;
188     for (int u = 0; u <= n; ++u) st[u] = u, flo[u].clear();
189     int w_max = 0;
190     for (int u = 1; u <= n; ++u)
191         for (int v = 1; v <= n; ++v) {
192             flo_from[u][v] = (u == v ? u : 0);
193             w_max = max(w_max, g[u][v].w);
194         }
195     for (int u = 1; u <= n; ++u) lab[u] = w_max;
196     while (matching()) ++n_matches;
197     for (int u = 1; u <= n; ++u)
198         if (match[u] && match[u] < u)
199             tot_weight += g[u][match[u]].w;
200     return make_pair(tot_weight, n_matches);
201 }
202 void add_edge(int ui, int vi, int wi) {
203     g[ui][vi].w = g[vi][ui].w = wi;
204 }
205 void init(int _n) {
206     n = _n;
207     for (int u = 1; u <= n; ++u)
208         for (int v = 1; v <= n; ++v)
209             g[u][v] = edge(u, v, 0);
210 }
211 };

```

3.10 prim

```

1  const int N = 2005;
2  int dis[N][N]; // input
3  int distfromtree[N];
4  for (int i = 1; i <= n; i++)
5      distfromtree[i] = dis[1][i];
6  int res = 0;
7  for (int i = 2; i <= n; i++) {
8      int mindist = INT_MAX, next = -1;
9      for (int j = 1; j <= n; j++) {
10         if (distfromtree[j] != 0 && distfromtree[j] < mindist) {
11             mindist = distfromtree[j];
12             next = j;
13         }
14     }
15     res += mindist;
16     for (int j = 1; j <= n; j++)
17         distfromtree[j] = min(distfromtree[j], dis[next][j]);
18 }

```

3.11 spfa

```

1  vector<int> G[N];
2  int spfa(int n, int s, int t) {
3      vector<int> dis(n, INT_MAX), cnt(n);
4      vector<bool> inq(n);
5      queue<int> q;
6      q.push(s);
7      dis[s] = 0;
8      inq[s] = true;
9      while (q.size()) {
10         int u = q.front();
11         q.pop();
12         cnt[u]++;
13         if (cnt[u] == n) // negative cycle
14             return INT_MAX;
15         inq[u] = false;
16         for (auto [v, w] : G[u]) {
17             if (dis[u] + w < dis[v]) {
18                 dis[v] = dis[u] + w;
19                 if (!inq[v]) {
20                     inq[v] = true;
21                     q.push(v);
22                 }
23             }
24         }
25     }
26     return dis[t];
27 }

```

4 math

4.1 big number

```

1  void init(string &a, string &b) {
2      while (a.size() < b.size()) a = '0' + a;
3      while (a.size() > b.size()) b = '0' + b;
4  }

```



```

5 int Compare(string &a, string &b) {
6     if (a < b) {
7         swap(a, b); return -1;
8     }
9     return 1;
10 }
11 bool del(string &a) {
12     if (a[0] == '0') {
13         a.erase(0, 1); return true;
14     }
15     return false;
16 }
17 void del_all_zero(string &a) {
18     while (del(a));
19 }
20 string add(string a, string b) {
21     init(a, b);
22     a = '0' + a; b = '0' + b;
23     for (int i = a.size() - 1; i >= 0; i--) {
24         int p = a[i] - '0', q = b[i] - '0';
25         if (p + q >= 10) {
26             a[i - 1] = '0' + (a[i - 1] - '0' + 1);
27             a[i] = '0' + (p + q - 10);
28         } else {
29             a[i] = '0' + (p + q);
30         }
31     }
32     del(a);
33     return a;
34 }
35 string sub(string a, string b) {
36     init(a, b);
37     if (a == b) return "0";
38     int flag = Compare(a, b);
39     for (int i = a.size() - 1; i >= 0; i--) {
40         int p = a[i] - '0', q = b[i] - '0';
41         if (p < q) {
42             a[i - 1] = '0' + (a[i - 1] - '0' - 1);
43             a[i] = '0' + (p - q + 10);
44         } else {
45             a[i] = '0' + (p - q);
46         }
47     }
48     del_all_zero(a);
49     if (flag == -1) a = "-" + a;
50     return a;
51 }
52 string mul(string a, string b) {
53     string res = "0";
54     init(a, b);
55     Compare(a, b);
56     del_all_zero(b);
57     for (int i = b.size() - 1; i >= 0; i--) {
58         int x = b[i] - '0';
59         if (i != b.size() - 1) a = a + '0';
60         for (int i = 0; i < x; i++)
61             res = add(a, res);
62     }
63     return res;
64 }
65 string div(string a, string b) {
66     init(a, b);
67     if (a < b) return "0";
68     del_all_zero(b);
69     string res = "0", restmp = "1", tmp = b;
70     for (int i = 0; i < (a.size() - b.size()); i++) {
71         restmp += '0';
72         tmp += '0';
73     }
74     init(a, b);
75     while (a >= b) {
76         init(a, tmp);
77         if (a >= tmp) {
78             a = sub(a, tmp);
79             res = add(res, restmp);
80         } else {
81             restmp.erase(restmp.size() - 1);
82             tmp.erase(tmp.size() - 1);
83         }
84         init(a, b);
85     }
86     return res;
87 }
88 signed main() {
89     string x, y, ch;
90     cin >> x >> ch >> y;
91     string ans;
92     if (ch[0] == '+') ans = add(x, y);
93     else if (ch[0] == '-') ans = sub(x, y);
94     else if (ch[0] == '*') ans = mul(x, y);
95     else ans = div(x, y);
96     cout << ans << "\n";
97     return 0;
98 }

```

4.2 burnside lemma

$$\sum_{k=1}^n \frac{c(k)}{n}$$

- n : 有多少種置換方式 (例如: 旋轉方式)
- $c(k)$: 所有可能中, 經過 k 次旋轉後, 仍不會和別人相同方式的數量

4.3 catalan

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

4.4 count prime

```

1 using namespace std;
2 // Count the number of primes not greater than N (N <= 10^11)
3 int primeCount(const int n) {
4     if (n <= 1) { return 0; }
5     if (n == 2) { return 1; }
6     const int v = sqrtl(n);
7     int s = (v + 1) / 2;
8     vector<int> smalls(s), roughs(s), skip(v + 1), larges(s);
9     iota(smalls.begin(), smalls.end(), 0);
10    for (int i = 0; i < s; i++) {
11        roughs[i] = 2 * i + 1;
12        larges[i] = (n / roughs[i] - 1) / 2;
13    }
14    const auto half = [](int n) -> int { return (n - 1) >> 1; };
15    int pc = 0;
16    for (int p = 3; p <= v; p += 2) {
17        if (skip[p]) { continue; }
18        int q = p * p;
19        if (1LL * q * q > n) { break; }
20        skip[p] = true;
21        for (int i = q; i <= v; i += 2 * p) skip[i] = true;
22        int ns = 0;
23        for (int k = 0; k < s; k++) {
24            int i = roughs[k];
25            if (skip[i]) { continue; }
26            int d = 1LL * i * p;
27            larges[ns] = larges[k] - (d <= v ? larges[smalls[d / 2] - pc] : smalls[half(n / d)] + pc;
28            roughs[ns++] = i;
29        }
30        s = ns;
31        for (int i = half(v), j = v / p - 1 | 1; j >= p; j -= 2) {
32            int c = smalls[j / 2] - pc;
33            for (int e = j * p / 2; i >= e; i--) { smalls[i] -= c; }
34        }
35        pc++;
36    }
37    larges[0] += 1LL * (s + 2 * (pc - 1)) * (s - 1) / 2;
38    for (int k = 1; k < s; k++) { larges[0] -= larges[k]; }
39    for (int l = 1; l < s; l++) {
40        int q = roughs[l];
41        int M = n / q;
42        int e = smalls[half(M / q)] - pc;
43        if (e <= 1) { break; }
44        int t = 0;
45        for (int k = 1 + 1; k <= e; k++) { t += smalls[half(M / roughs[k])]; }
46        larges[0] += t - 1LL * (e - 1) * (pc + 1 - 1);
47    }
48    return larges[0] + 1;
49 }

```

4.5 CRT coprime

```

1 vector<int> a, m;
2 int extgcd(int a, int b, int& x, int& y) {
3     if (b == 0) {
4         x = 1, y = 0;
5         return a;
6     }
7     int ret = extgcd(b, a % b, y, x);
8     y -= a / b * x;
9     return ret;
10 }
11 // n = 有幾個式子 · 求解 x ≡ a_i (mod m_i)
12 int CRT(int n, vector<int>& a, vector<int>& m) {
13     int p = 1, ans = 0;
14     vector<int> M(n), inv_M(n);
15     for (int i = 0; i < n; i++)
16         p *= m[i];
17     for (int i = 0; i < n; i++) {
18         M[i] = p / m[i];
19         extgcd(M[i], m[i], inv_M[i], tmp);
20         ans += a[i] * inv_M[i] * M[i];
21         ans %= p;

```

```

22 }
23 return (ans % p + p) % p;
24 }

```

4.6 CRT coprime

n = 有幾個式子 · 求解 $x \equiv a_i \pmod{m_i}$

4.7 CRT not coprime

```

1 int extgcd(int a, int b, int& x, int& y) {
2     if (b == 0) {
3         x = 1, y = 0;
4         return a;
5     }
6     int ret = extgcd(b, a % b, y, x);
7     y -= a / b * x;
8     return ret;
9 }
10 // 對於方程組的式子兩兩求解 · return {是否有解, {a, m}}
11 pair<bool, pair<int, int>> CRT(int a1, int m1, int a2, int m2)
12 {
13     int g = __gcd(m1, m2);
14     if ((a2 - a1) % g != 0)
15         return {0, {-1, -1}};
16     int x, y;
17     extgcd(m1, m2, x, y);
18     x = (a2 - a1) * x / g; // 兩者不能相反
19     a1 = x * m1 + a1;
20     m1 = m1 * m2 / g;
21     a1 = (a1 % m1 + m1) % m1;
22     return {1, {a1, m1}};
23 }

```

4.8 det

```

1 ll Det(vector<vector<ll>> a) {
2     int n = a.size();
3     ll det = 1;
4     for (int i = 0; i < n; ++i) {
5         if (!a[i][i]) {
6             det = -det;
7             if (det < 0) det += mod;
8             for (int j = i + 1; j < n; ++j) if (a[j][i]) {
9                 swap(a[j], a[i]);
10                break;
11            }
12            if (!a[i][i]) return 0;
13        }
14        det = det * a[i][i] % mod;
15        ll mul = mpow(a[i][i], mod - 2);
16        for (int j = 0; j < n; ++j)
17            a[i][j] = a[i][j] * mul % mod;
18        for (int j = 0; j < n; ++j) if (i ^ j) {
19            ll mul = a[j][i];
20            for (int k = 0; k < n; ++k) {
21                a[j][k] -= a[i][k] * mul % mod;
22                if (a[j][k] < 0) a[j][k] += mod;
23            }
24        }
25    }
26    return det;
27 }

```

4.9 extgcd

```

1 // a * p.first + b * p.second = gcd(a, b)
2 pair<ll, ll> extgcd(ll a, ll b) {
3     pair<ll, ll> res, tmp;
4     ll f = 1, g = 1;
5     if (a < 0) a *= -1, f *= -1;
6     if (b < 0) b *= -1, g *= -1;
7     if (b == 0) return {f, 0};
8     tmp = extgcd(b, a % b);
9     res.first = tmp.second * f;
10    res.second = (tmp.first - tmp.second * (a / b)) * g;
11    return res;
12 }

```

4.10 factorize

```

1 using i64 = long long;
2 using i128 = __int128_t;
3 struct Factorize { // 質因數分解
4     i64 fmul(i64 a, i64 b, i64 p) {
5         return (i128)a * b % p;
6     }
7     i64 fpow(i64 a, i64 b, i64 p) {
8         i64 res = 1;
9         for (; b >= 1; a = fmul(a, a, p))
10             if (b & 1) res = fmul(res, a, p);
11         return res;
12     }
13     bool check(i64 a, i64 u, i64 n, int t) {
14         a = fpow(a, u, n);
15         if (a == 0 or a == 1 or a == n - 1)
16             return true;
17         for (int i = 0; i < t; i++) {
18             a = fmul(a, a, n);
19             if (a == 1) return false;
20             if (a == n - 1) return true;
21         }
22         return false;
23     };
24     bool isPrime(i64 n) {
25         constexpr array<i64, 7> magic{2, 235, 9375, 28178, 450775,
26             9780504, 1795265022};
27         // for int: {2, 7, 61}
28         if (n < 2) return false;
29         if (n % 2 == 0) return n == 2;
30         i64 u = n - 1;
31         int t = 0;
32         while (u % 2 == 0) u >>= 1, t++;
33         for (auto v : magic) if (!check(v, u, n, t)) return false;
34         return true;
35     }
36     i64 PollardRho(i64 n) { // return non-trivial factor of n
37         if (n % 2 == 0) return 2;
38         i64 x = 2, y = 2, d = 1, p = 1;
39         auto f = [(i64 x, i64 n, i64 p) -> i64 {
40             return ((i128)x * x % n + p) % n;
41         }];
42         while (true) {
43             x = f(x, n, p);
44             y = f(f(y, n, p), n, p);
45             d = gcd(abs(x - y), n);
46             if (d != n and d != 1) return d;
47             if (d == n) ++p;
48         }
49     }
50     i64 primeFactor(i64 n) {
51         return isPrime(n) ? n : primeFactor(PollardRho(n));
52     }
53     void solve() {
54         i64 n;
55         cin >> n;
56         vector<i64> ans;
57         while (n > 1) {
58             i64 p = ftr.primeFactor(n);
59             do {
60                 ans.push_back(p);
61                 n /= p;
62             } while (n % p == 0);
63         }
64         sort(ans.begin(), ans.end());
65         cout << ans.size() << ' ';
66         for (int i : ans) cout << i << ' ';
67         cout << '\n';
68     }
69 }

```

4.11 fraction

```

1 struct frac {
2     int a, b;
3     frac(int _a = 0, int _b = 1) : a(_a), b(_b) {
4         int g = __gcd(a, b);
5         a /= g; b /= g;
6         if (b < 0) { a *= -1; b *= -1; }
7     }
8 };
9 frac operator+(frac x, frac y) { return frac(x.a * y.b + y.a *
10     x.b, x.b * y.b); }
11 frac operator-(frac x, frac y) { return frac(x.a * y.b - y.a *
12     x.b, x.b * y.b); }
13 frac operator*(frac x, frac y) { return frac(x.a * y.a, x.b * y
14     .b); }
15 frac operator/(frac x, frac y) { return frac(x.a * y.b, x.b * y
16     .a); }
17 bool operator>(frac x, frac y) { return x.a * y.b > y.a * x.b;
18 }

```

```

14 bool operator<(frac x, frac y) { return x.a * y.b < y.a * x.b;
    }
15 bool operator>=(frac x, frac y) { return x.a * y.b >= y.a * x.b
    ; }
16 bool operator<=(frac x, frac y) { return x.a * y.b <= y.a * x.b
    ; }
17 bool operator==(frac x, frac y) { return x.a * y.b == y.a * x.b
    ; }
18 frac abs(frac x) { return frac(abs(x.a), abs(x.b)); }
19 ostream& operator<<(ostream &os, const frac& x) { os << x.a <<
    "/" << x.b; return os; }

```

4.12 gaussian elimination

```

1 #define int long long
2 const int N = 25;
3 frac co[N][N];
4 // SIZE = n * (n + 1)
5 // a_1 x_1 + a_2 x_2 + ... + a_n x_n = a_{n+1}
6 void gaussian(int n) {
7     for (int i = 1; i <= n; i++) {
8         if (co[i][i].a == 0 && co[i][i].b == 1) { // swap
9             int swapid = -1;
10            for (int id = i + 1; id <= n; id++) {
11                if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
12                    swapid = id; break;
13                }
14            }
15            if (swapid == -1) continue;
16            else {
17                for (int j = 1; j <= n + 1; j++)
18                    swap(co[i][j], co[swapid][j]);
19            }
20        }
21        for (int id = i + 1; id <= n; id++) {
22            if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
23                frac mul = co[id][i] / co[i][i];
24                for (int j = 1; j <= n + 1; j++)
25                    co[id][j] = co[id][j] - co[i][j] * mul;
26            }
27        }
28    }
29    for (int i = n; i >= 1; i--) {
30        if (! (co[i][i].a == 0 && co[i][i].b == 1)) {
31            co[i][n + 1] = co[i][n + 1] / co[i][i];
32            co[i][i] = {1, 1};
33        } else {
34            continue;
35        }
36        for (int id = 1; id < i; id++) {
37            if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
38                frac mul = co[id][i] / co[i][i];
39                for (int j = 1; j <= n + 1; j++)
40                    co[id][j] = co[id][j] - co[i][j] * mul;
41            }
42        }
43    }
44    bool check_inf = false, check_no = false;
45    for (int i = 1; i <= n; i++) {
46        bool allzro = true;
47        for (int j = 1; j <= n; j++)
48            if (! (co[i][j].a == 0 && co[i][j].b == 1))
49                allzro = false;
50        if (allzro) {
51            if (! (co[i][n + 1].a == 0 && co[i][n + 1].b == 1))
52                check_no = true;
53            else
54                check_inf = true;
55        }
56    }
57    if (check_no) {
58        cout << "no solutions\n";
59    } else if (check_inf) {
60        cout << "infinite many solutions\n";
61    } else {
62        for (int i = 1; i <= n; i++)
63            cout << "X" << i << " = " << co[i][n + 1] << '\n';
64    }
65 }

```

4.13 josephus

```

1 // 有 n 個人，第偶數個報數的人被刪掉，問第 k 個被踢掉的是誰
2 int solve(int n, int k) {
3     if (n == 1) return 1;
4     if (k <= (n + 1) / 2) {
5         if (2 * k > n)
6             return 2 * k % n;
7         else
8             return 2 * k;

```

```

9     } else {
10        int res = solve(n / 2, k - (n + 1) / 2);
11        if (n & 1)
12            return 2 * res + 1;
13        else
14            return 2 * res - 1;
15    }
16 }

```

4.14 lucas

```

1 // 對於很大的 C^n_m 對質數 p 取模，只要 p 不大就可以用
2 int Lucas(int n, int m, int p) {
3     if (m == 0) return 1;
4     return (C(n % p, m % p, p) * Lucas(n / p, m / p, p) % p);
5 }

```

4.15 math block

```

1 // the sum of floor(n / i) for i = 1 ~ n
2 int f(int n) {
3     int ans = 0;
4     int l = 1, r = 0;
5     while (l <= n) {
6         r = n / (n / l);
7         ans += (r - l + 1) * (n / l);
8         l = r + 1;
9     }
10    return ans;
11 }
12 // the sum of ceil(n / i) for i = 1 ~ n
13 int f(int n) {
14     int ans = 0;
15     int l = 1, r = 0;
16     while (l < n) {
17         r = (n - 1) / ((n - 1) / l);
18         ans += (r - l + 1) * ((n + 1 - 1) / l);
19         l = r + 1;
20     }
21     if (l == n) ans += (n + 1 - 1) / l;
22     return ans;
23 }

```

4.16 miller rabin

```

1 struct Miller_Rabin {
2     static constexpr uint32_t ws32[3] = {2, 7, 61};
3     static constexpr uint64_t ws64[7] = {2, 325, 9375, 28178,
4         450775, 9780504, 1795265022};
5     template <class uint>
6     static constexpr uint fast_pow(uint b, uint p, uint mod) {
7         using Uint = conditional_t<is_same_v<uint, uint32_t>,
8             uint64_t, __uint128_t>;
9         uint ret{1};
10        for (; p; p >>= 1) {
11            if (p & 1) ret = Uint{ret} * b % mod;
12            b = Uint{b} * b % mod;
13        }
14        return ret;
15    }
16    template <class uint, class = enable_if_t<is_unsigned_v<uint>
17        >>>
18    static constexpr bool is_prime(uint n) {
19        const auto& witness = []() -> const auto& {
20            if constexpr (is_same_v<uint, uint32_t>) return ws32;
21            else return ws64;
22        }();
23        using Uint = conditional_t<is_same_v<uint, uint32_t>,
24            uint64_t, __uint128_t>;
25        if (n < 3 || !(n & 1)) return n == 2;
26        auto u = n - 1, t = (uint) __builtin_ctzll(u);
27        u >>= t;
28        for (auto x : witness) {
29            auto a = x % n;
30            if (a == 0 || a == 1 || a == n - 1) continue;
31            auto v = fast_pow(a, u, n);
32            uint i = 0;
33            for (; v != 1 && v != n - 1 && i < t; i++) v = Uint{v} *
34                v % n;
35            if (v != n - 1 && i > 0) return false;
36        }
37        return true;
38    }
39    template <class I, class = enable_if_t<is_integral_v<I>>>
40    constexpr bool operator()(I x) const {
41        if constexpr (is_signed_v<I>) if (x < 0) return false;
42        using U = make_unsigned_t<I>;

```



```

12 int calc(int x) { // 小於等於 x 的數量
13     if (x == 0) return 0;
14     int arr[20] = {}, len = 0;
15     while (x > 0) arr[++len] = x % 10, x /= 10;
16     // 計算 Len 位數 的數量
17     int cnt = 0, pre = -1;
18     for (int idx = len; idx >= 1; idx--) {
19         for (int now = (idx == len); now < arr[idx]; now++)
20             if (now != pre)
21                 cnt += dp[idx][now];
22         if (arr[idx] == pre) break;
23         pre = arr[idx];
24         if (idx == 1) cnt++; // 特判
25     }
26     // 計算 小於 Len 位數 的數量
27     for (int i = 1; i < len; i++)
28         for (int j = 1; j <= 9; j++)
29             cnt += dp[i][j];
30     return cnt;
31 }

```

5.4 dinic

```

1 struct Flow {
2     struct Edge {
3         // rc : residual capacity
4         int u, rc, rev;
5     };
6     vector<vector<Edge>> G;
7     vector<int> dis, it;
8     Flow(int n) : G(n), dis(n), it(n) {}
9     void add(int v, int u, int c) {
10         G[v].push_back({u, c, sz(G[u])});
11         G[u].push_back({v, 0, sz(G[v]) - 1});
12     }
13     int dfs(int v, int t, int f) {
14         if (v == t || f == 0) return f;
15         for (int &i = it[v]; i < sz(G[v]); i++) {
16             auto &[u, rc, rev] = G[v][i];
17             if (dis[u] != dis[v] + 1) continue;
18             int res = dfs(u, t, min(f, rc));
19             if (res <= 0) continue;
20             rc -= res;
21             G[u][rev].rc += res;
22             return res;
23         }
24         return 0;
25     }
26     int flow(int s, int t) {
27         int ans = 0;
28         for (int l = 30; l >= 0; l--) while (true) {
29             fill(all(dis), INT_MAX);
30             queue<int> Q;
31             Q.push(s);
32             dis[s] = 0;
33             while (!Q.empty()) {
34                 int v = Q.front(); Q.pop();
35                 for (auto [u, rc, rev] : G[v]) {
36                     if ((rc >> 1) <= 0 || dis[u] < INT_MAX) continue;
37                     dis[u] = dis[v] + 1;
38                     Q.push(u);
39                 }
40             }
41             if (dis[t] == INT_MAX) break;
42             fill(all(it), 0);
43             while (true) {
44                 int res = dfs(s, t, INT_MAX);
45                 if (res <= 0) break;
46                 ans += res;
47             }
48         }
49         return ans;
50     }
51 }
52 };

```

5.5 geometry

```

1 double eps = 0.000001;
2 pdd operator + (pdd a, pdd b) { return {a.X + b.X, a.Y + b.Y}; }
3 pdd operator - (pdd a, pdd b) { return {a.X - b.X, a.Y - b.Y}; }
4 double dot(pdd a, pdd b) { return a.X * b.X + a.Y * b.Y; }
5 double cross(pdd a, pdd b) { return a.X * b.Y - a.Y * b.X; }
6 int sign(double x) { return (fabs(x) < eps ? 0 : (x > 0 ? 1 : -1)); }
7 int ori(pdd a, pdd b, pdd c) { return sign(cross(b - a, c - a)); }
8 // c is between a and b

```

```

9 bool btw(pdd a, pdd b, pdd c) {
10     return (sign(cross(a - c, b - c)) == 0 && sign(dot(a - c, b - c)) <= 0);
11 }
12 // ab is touch cd
13 bool banana(pdd a, pdd b, pdd c, pdd d) {
14     int a123 = ori(a, b, c), a124 = ori(a, b, d);
15     int a341 = ori(c, d, a), a342 = ori(c, d, b);
16     if (btw(a, b, c) || btw(a, b, d) || btw(c, d, a) || btw(c, d, b))
17         return true;
18     return (a123 * a124 < 0 && a341 * a342 < 0);
19 }
20 vector<pdd> convex_hull(vector<pdd> pts) {
21     sort(pts.begin(), pts.end());
22     pts.resize(unique(pts.begin(), pts.end()) - pts.begin());
23     vector<pdd> hull(1, pts[0]);
24     for (int t = 0; t < 2; t++) {
25         int sz = hull.size();
26         for (int i = 1; i < (int)pts.size(); i++) {
27             while ((int)hull.size() > sz && ori(hull[(int)hull.size() - 2], hull.back(), pts[i]) < 0)
28                 hull.pop_back();
29             hull.push_back(pts[i]);
30         }
31         reverse(pts.begin(), pts.end());
32     }
33     hull.pop_back();
34     return hull;
35 }
36 double area(vector<pdd> pt) {
37     double cnt = 0.0;
38     for (int i = 0; i < (int)pt.size() - 1; i++)
39         cnt += cross(pt[i], pt[i + 1]);
40     cnt += cross(pt[(int)pt.size() - 1], pt[0]);
41     return fabs(cnt);
42 }
43 void solve() {
44     int n;
45     cin >> n;
46     vector<pdd> pts;
47     pts.resize(n);
48     for (int i = 0; i < n; i++)
49         cin >> pts[i].X >> pts[i].Y;
50     int ans = area(convex_hull(pts));
51     if (ans % 2 == 1)
52         cout << ans / 2 << ".5\n";
53     else
54         cout << ans / 2 << ".0\n";
55 }

```

5.6 line container

```

1 // only works for integer coordinates!! maintain max
2 struct Line {
3     mutable ll a, b, p;
4     bool operator<(const Line &rhs) const { return a < rhs.a; }
5     bool operator<(ll x) const { return p < x; }
6 };
7 struct DynamicHull : multiset<Line, less<>> {
8     static const ll kInf = 1e18;
9     ll Div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
10    bool isect(iterator x, iterator y) {
11        if (y == end()) { x->p = kInf; return 0; }
12        if (x->a == y->a) x->p = x->b > y->b ? kInf : -kInf;
13        else x->p = Div(y->b - x->b, x->a - y->a);
14        return x->p >= y->p;
15    }
16    void addline(ll a, ll b) { // ax + b
17        auto z = insert({a, b, 0}); y = z++, x = y;
18        while (isect(y, z)) z = erase(z);
19        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
20        while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
21    }
22    ll query(ll x) {
23        auto l = *lower_bound(x);
24        return l.a * x + l.b;
25    }
26 };

```

5.7 mcmf

```

1 struct Flow {
2     struct Edge {
3         int u, rc, k, rv;
4     };
5     vector<vector<Edge>> G;
6     vector<int> par, par_eid;
7     Flow(int n) : G(n + 1), par(n + 1), par_eid(n + 1) {}
8     void add(int v, int u, int c, int k) {

```

```

9   G[v].push_back({u, c, k, (int)G[u].size()});
10  G[u].push_back({v, 0, -k, (int)G[v].size() - 1});
11
12  int spfa(int s, int t) {
13      fill(par.begin(), par.end(), -1);
14      vector<int> dis(par.size(), LONG_LONG_MAX);
15      vector<bool> in_q(par.size(), false);
16      queue<int> Q;
17      dis[s] = 0; in_q[s] = true;
18      Q.push(s);
19      while (!Q.empty()) {
20          int v = Q.front(); Q.pop();
21          in_q[v] = false;
22          for (int i = 0; i < (int)G[v].size(); i++) {
23              auto [u, rc, k, rv] = G[v][i];
24              if (rc > 0 && dis[v] + k < dis[u]) {
25                  dis[u] = dis[v] + k;
26                  par[u] = v;
27                  par_eid[u] = i;
28                  if (!in_q[u]) Q.push(u);
29                  in_q[u] = true;
30              }
31          }
32      }
33      return dis[t];
34  }
35  pair<int, int> flow(int s, int t) {
36      int fl = 0, cost = 0, d;
37      while ((d = spfa(s, t)) < LONG_LONG_MAX) {
38          int cur = LONG_LONG_MAX;
39          for (int v = t; v != s; v = par[v])
40              cur = min(cur, G[par[v]][par_eid[v]].rc);
41          fl += cur;
42          cost += d * cur;
43          for (int v = t; v != s; v = par[v]) {
44              G[par[v]][par_eid[v]].rc -= cur;
45              G[v][G[par[v]][par_eid[v]].rv].rc += cur;
46          }
47      }
48      return {fl, cost};
49  }
50  };

```

5.8 mo

```

1  struct MoSolver {
2      struct query {
3          int l, r, id;
4          bool operator < (const query &o) {
5              if (1 / C == o.l / C) return (1 / C) & 1 ? r > o.r : r <
6                  o.r;
7              return 1 / C < o.l / C;
8          }
9      };
10     int cur_ans;
11     vector<int> ans;
12     void add(int x) {
13         // do something
14     }
15     void sub(int x) {
16         // do something
17     }
18     vector<query> Q;
19     void add_query(int l, int r, int id) {
20         // [l, r)
21         Q.push_back({l, r, id});
22         ans.push_back(0);
23     }
24     void run() {
25         sort(Q.begin(), Q.end());
26         int pl = 0, pr = 0;
27         cur_ans = 0;
28         for (query &i : Q) {
29             while (pl > i.l)
30                 add(a[--pl]);
31             while (pr < i.r)
32                 add(a[pr++]);
33             while (pl < i.l)
34                 sub(a[pl++]);
35             while (pr > i.r)
36                 sub(a[--pr]);
37             ans[i.id] = cur;
38         }
39     };

```

5.9 number split

```

1  // dp[i][x] = 要將整數 x 拆成 i 堆的「組合數」
2  dp[i + 1][x + 1] += dp[i][x]; // 創造新的一堆
3  dp[i][x + i] += dp[i][x];    // 把每一堆都增加 1

```

5.10 sos dp

```

1  // 時間複雜度為  $O(n * 2^n)$ 
2  // 計算  $dp[i]$  =  $i$  所有 bit mask 子集的和
3  for (int i = 0; i < n; i++)
4      for (int mask = 0; mask < (1 << n); mask++)
5          if ((mask >> i) & 1)
6              dp[mask] += dp[mask ^ (1 << i)];

```

5.11 subset

```

1  for (int i = 0; i < (1 << n); i++){
2      for (int m = i; m; m = (m - 1) & i){
3          // m 是 i 的非空子集
4      }
5  }

```

5.12 xor basis

```

1  // 給一數字集合 S，求能 XOR 出多少數字
2  // 答案等於  $2^{\text{basis 的大小}}$ 
3  vector<int> basis;
4  void add_vector(int x) {
5      for (int v : basis) x = min(x, x ^ v);
6      if (x) basis.push_back(x);
7  }
8  // 給一數字集合 S，求能不能 XOR 出 x
9  bool check(int x) {
10     for (int v : basis) x = min(x, x ^ v);
11     return x;
12 }
13 // 給一數字集合 S，求 XOR 出最大的數字
14 int get_max() {
15     int ans = 0;
16     for (auto v : basis) ans = max(ans, ans ^ v);
17     return ans;
18 }

```

6 string

6.1 hash

```

1  const int p = 1e6 + 3;
2  const int mod = 1e9 + 7;
3  int hash(string &s) {
4      int res = 0;
5      for (int i = 0; i < s.size(); i++) {
6          res *= p;
7          res += s[i] - 'a' + 1;
8          res %= mod;
9      }
10     return res;
11 }
12 int pow_p[1000005];
13 vector<int> rh;
14 void build(string &s) {
15     rh.resize(s.size() + 1);
16     pow_p[0] = 1;
17     for (int i = 1; i <= s.size(); i++)
18         pow_p[i] = (pow_p[i - 1] * p) % mod;
19     rh[0] = s[0] - 'a' + 1;
20     for (int i = 1; i < s.size(); i++) {
21         rh[i] = rh[i - 1] * p + (s[i] - 'a' + 1);
22         rh[i] %= mod;
23     }
24 }
25 int query(int l, int r) {
26     int res = rh[r] - (l > 0 ? rh[l - 1] * pow_p[r - l + 1] : 0);
27     res = (res % mod + mod) % mod;
28     return res;
29 }

```


6.2 kmp

```

1 vector<int> F;
2 void build_failure_function(string &s) {
3     F.clear();
4     F.resize(s.size(), -1);
5     for (int i = 1; i < s.size(); i++) {
6         int j = F[i - 1];
7         while (j != -1 && s[i] != s[j + 1])
8             j = F[j];
9         if (s[i] == s[j + 1])
10            F[i] = j + 1;
11    }
12 }
13 void KMP_matching(string &a, string &b) {
14     // i -> a 的指針, j -> b 的指針
15     for (int i = 0, j = -1; i < a.size(); i++) {
16         while (j != -1 && a[i] != b[j + 1]) // 匹配失敗
17             j = F[j];
18         if (a[i] == b[j + 1]) // 匹配成功
19             j++;
20         if (j + 1 == b.size()) { // 找到了, 當作匹配失敗 重新匹配
21             // 找下一個
22             cout << "found a matching start at " << i - j << "\n";
23             j = F[j];
24         }
25    }
26 }

```

6.3 manacher

```

1 string s;
2 int extend(int l, int r, int N) {
3     int i = 0;
4     while (l - i >= 0 && r + i < N && s[l - i] == s[r + i])
5         i++;
6     return i;
7 }
8 int Longest_Palindromic_Substring(string &t) {
9     int N = t.length();
10    s.resize(2 * N + 1, '$');
11    for (int i = 0; i < N; i++)
12        s[2 * i + 1] = t[i];
13    N = 2 * N + 1;
14    vector<int> res;
15    res.resize(N, 0);
16    res[0] = 1;
17    for (int i = 1, mid = 0, R = 0; i < N; i++) {
18        int j = mid - (i - mid);
19        int lst = R + 1 - i;
20        if (i > R) {
21            res[i] = extend(i, i, N);
22            mid = i;
23            R = i + res[i] - 1;
24        } else if (res[j] == lst) {
25            res[i] = lst + extend(i - R, i + R, N);
26            mid = i;
27            R = i + res[i] - 1;
28        } else {
29            res[i] = min(res[j], lst);
30        }
31    }
32    int mx = -1, idx = -1;
33    for (int i = 0; i < N; i++) {
34        if (res[i] > mx) {
35            mx = res[i];
36            idx = i;
37        }
38    }
39    cout << "Longest Palindromic Substring Length is " << (mx + 1) / 2 << "\n";
40    for (int i = idx - res[idx] + 1; i <= idx + res[idx] + 1; i++)
41        if (i & 1)
42            cout << s[i];
43    cout << "\n";
44 }

```

6.4 string trie

```

1 struct node {
2     node* link[26] = {nullptr};
3     bool flag = false;
4     bool contain_key(char ch) { return (link[ch - 'a'] != nullptr); }
5     void put(char ch, node* _node) { link[ch - 'a'] = _node; }
6     node* get(char ch) { return link[ch - 'a']; }
7     void set_end() { flag = true; }
8     bool is_end() { return flag; }

```

```

9 };
10 struct Trie {
11     node* root;
12     Trie() { root = new node(); }
13     void insert(string s) { // insert a string
14         node* now = root;
15         for (int i = 0; i < (int)s.size(); i++) {
16             if (!now->contain_key(s[i])) now->put(s[i], new node());
17             now = now->get(s[i]);
18         }
19         now->set_end();
20     }
21     bool query(string s) { // return if there is a word of the
22         // given string
23         node* now = root;
24         for (int i = 0; i < (int)s.size(); i++) {
25             if (!now->contain_key(s[i])) return false;
26             now = now->get(s[i]);
27         }
28         return now->is_end();
29     }
30     // return if there is any word that start with the given
31     // prefix
32     bool query_prefix(string s) {
33         node* now = root;
34         for (int i = 0; i < (int)s.size(); i++) {
35             if (!now->contain_key(s[i])) return false;
36             now = now->get(s[i]);
37         }
38         return true;
39     }
40 };

```

6.5 xor trie

```

1 struct node {
2     node* link[2];
3     bool contain_key(int val) { return (link[val] != nullptr); }
4     node* get(int val) { return link[val]; }
5     void put(int val, node* _node) { link[val] = _node; }
6 };
7 struct Trie {
8     node* root;
9     Trie() { root = new node(); }
10    void update(int x) {
11        node* now = root;
12        for (int i = 30; i >= 0; i--) {
13            if (x & (1 << i)) {
14                if (!now->contain_key(1)) now->put(1, new node());
15                now = now->get(1);
16            } else {
17                if (!now->contain_key(0)) now->put(0, new node());
18                now = now->get(0);
19            }
20        }
21    }
22    int query(int x) { // query MAX XOR with number x
23        node* now = root;
24        int mx = 0;
25        for (int i = 30; i >= 0; i--) {
26            int nowbit = (x >> i) & 1;
27            int target = nowbit ^ 1;
28            if (now->contain_key(target)) {
29                mx += (1 << i);
30                now = now->get(target);
31            } else {
32                now = now->get(nowbit);
33            }
34        }
35        return mx;
36    }
37 };

```

6.6 z

```

1 vector<int> Z;
2 void calculate_Z(string &s) {
3     Z.clear();
4     Z.resize(s.size(), 0);
5     int l = 0, r = 0;
6     for (int i = 1; i < s.size(); i++) {
7         if (i <= r) // 估算下界
8             Z[i] = min(Z[i - l], r - i + 1);
9         while (i + Z[i] < s.size() && s[i + Z[i]] == s[Z[i]]) // 暴力檢查 Z(i) 是否可以變更大
10            Z[i]++;
11         if (i + Z[i] - 1 > r) { // 更新 "看到最右邊的區間 [l, r]"
12             l = i; r = i + Z[i] - 1;
13         }
14     }

```



```

15 }
16 void Z_matching(string &a, string &b) {
17     string res = "";
18     int i = 0;
19     for (char ch : b)
20         res[i++] = ch;
21     res[i++] = '_';
22     for (char ch : a)
23         res[i++] = ch;
24     calculate_Z(res);
25     for (int i = 0; i < Z.size(); i++) {
26         if (Z[i] == b.size())
27             cout << "found a matching start at " << i - b.size() - 1
28                 << "\n";
29     }
}

```

7 tree

7.1 hld

```

1  const int N = 100005;
2  vector<int> G[N];
3  struct HLD {
4      vector<int> pa, sz, depth, mxson, topf, id;
5      int n, idcnt = 0;
6      HLD(int _n) : n(_n), pa(_n + 1), sz(_n + 1), depth(_n + 1),
7                  mxson(_n + 1), topf(_n + 1), id(_n + 1) {}
8      void dfs1(int v = 1, int p = -1) {
9          pa[v] = p; sz[v] = 1; mxson[v] = 0;
10         depth[v] = (p == -1 ? 0 : depth[p] + 1);
11         for (int u : G[v]) {
12             if (u == p) continue;
13             dfs1(u, v);
14             sz[v] += sz[u];
15             if (sz[u] > sz[mxson[v]]) mxson[v] = u;
16         }
17     }
18     void dfs2(int v = 1, int top = 1) {
19         id[v] = ++idcnt;
20         topf[v] = top;
21         if (mxson[v]) dfs2(mxson[v], top);
22         for (int u : G[v]) {
23             if (u == mxson[v] || u == pa[v]) continue;
24             dfs2(u, u);
25         }
26     }
27     // query 為區間資料結構
28     int path_query(int a, int b) {
29         int res = 0;
30         while (topf[a] != topf[b]) { // 若不在同一條鍊上
31             if (depth[topf[a]] < depth[topf[b]]) swap(a, b);
32             res = max(res, query(id[topf[a]], id[a])); // query : l =
33                 id[topf[a]], r = id[a]
34             a = pa[topf[a]];
35         }
36         // 此時已在同一條鍊上
37         if (depth[a] < depth[b]) swap(a, b);
38         res = max(res, query(id[b], id[a])); // query : l = id[b],
39             r = id[a]
40         return res;
41     }
42 }

```